

Persona-stable NPCs on a 1.1B model: questions the panel will ask

Each question has a **short answer** to say out loud, then the detail behind it. Every number here comes from the repository (paper/NUMBERS.md, training configs, result files). Where something is not measured, the document says so. Don't fill those gaps with guesses in the review.

Base model: TinyLlama-1.1B-Chat 8 persona adapters · $r=16$ · $\alpha=32$

Dataset: 1,160 pairs Updated 15 Sep 2026

MODEL & COMPRESSION

Q1

What is the difference between LoRA and QLoRA?

SHORT ANSWER

Both freeze the base model and train small low-rank matrices added to it. LoRA keeps the frozen base in 16-bit; QLoRA stores the frozen base in 4-bit (NF4) during training, so it fits in much less GPU memory. The trained adapter is the same kind of object either way.

LoRA

- Base weights frozen, in 16/32-bit
- Learns $W + B \cdot A$, with B and A of rank r
- Training memory: base at 2 bytes/param plus activations
- Hu et al., 2021

QLoRA

- Base weights frozen, **quantized to 4-bit NF4**
- Same $B \cdot A$ adapter, computed in bf16
- Adds double quantization and paged optimizers
- Dettmers et al., 2023

The key idea: LoRA decides *what is trained* (a small low-rank update). The Q in QLoRA decides *how the frozen part is stored while training*. Gradients flow through the 4-bit base into the 16-bit adapter.

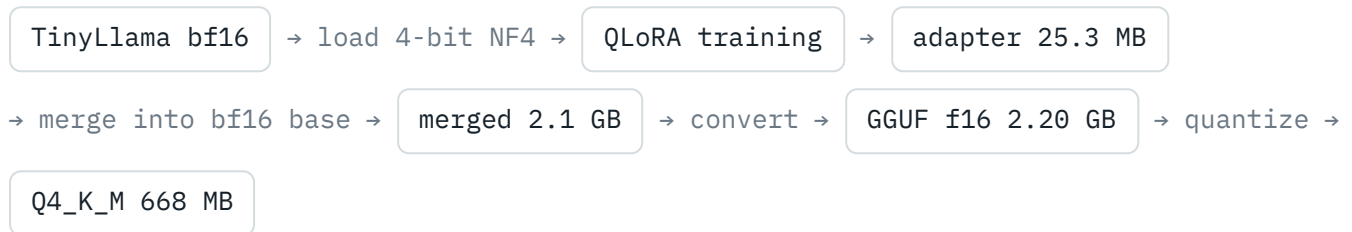
Ours: QLoRA, from `training/train_adapter.py`: `load_in_4bit=True`,
`bnb_4bit_quant_type="nf4"`, `bnb_4bit_compute_dtype=bfloat16`,
`bnb_4bit_use_double_quant=True`.

How is LoRA "converted" to QLoRA?

SHORT ANSWER

It isn't converted. QLoRA is a way of *training* a LoRA adapter: load the base model in 4-bit and attach the adapter. For serving we go the other way: merge the adapter into a full-precision copy of the base, then quantize that merged model to 4-bit GGUF.

Our full path, from training to the game:



1. **Train (QLoRA):** base loaded in 4-bit; only the adapter (B, A matrices) learns.
2. **Merge** (training/merge_lora.py): base reloaded in **bf16**, **not 4-bit**, then `merge_and_unload()` folds $B \cdot A$ into the weights. Merging into an already-quantized base would add quantization error on top of the adapter.
3. **Convert** to GGUF f16 (llama.cpp format).
4. **Quantize** to Q4_K_M (training/quantize_gguf.py), which llama.cpp runs on CPU in about 0.3–0.45 s per reply.

Common mix-up. "4-bit" appears twice for different reasons: NF4 during *training* (bitsandbytes, QLoRA) and Q4_K_M at *inference* (llama.cpp). They are different quantization schemes applied at different stages.

Q3

If a fine-tuned model is ~12 GB, how does it become ~2 GB? What does LoRA ignore, and does accuracy go down?

SHORT ANSWER

Three separate things shrink the model. A full fine-tune saves a whole new model, and our archived run's output was 11.86 GB, likely including training state. LoRA saves only the *change*: 12.6 M parameters, 25.3 MB per character. The base model itself is 2.2 GB in 16-bit, and quantizing it to 4-bit brings it to 668 MB. LoRA doesn't skip any data. It limits the *shape* of the weight update to rank 16 and leaves most weights untouched. For a style/persona task that costs little; we have not measured the gap on our own data.

Size on disk · logarithmic scale · police officer adapter

Full fine-tune output

archived run, measured

11.86 GB

Weights only, fp32

1.1 B × 4 bytes, computed

~4.4 GB

Merged model, 16-bit

GGUF f16, measured

2.20 GB

Merged model, Q4_K_M

what the game runs

668 MB

LoRA adapter only

r=16, 7 modules, measured

25.3 MB

10 MB

100 MB

1 GB

10 GB

100 GB

■ measured file ■ computed

Where "12 GB → 2 GB" actually comes from:

- **Full fine-tune (11.86 GB):** every one of the 1.1 B weights gets a new value, so the whole model is saved again, along with the state training keeps next to it. The weights alone are only 2.2 GB (16-bit) to 4.4 GB (32-bit), so most of the 11.86 GB is very likely training state saved with the run, such as optimizer moments. We haven't re-checked what the archived folder contains, so say "likely".

- **The ~2 GB is the merged model** (base + adapter folded in, 16-bit). LoRA alone doesn't make the model 2 GB. It makes the *per-character file* 25.3 MB, so eight characters cost 8×25 MB instead of 8×2.2 GB.
- **668 MB** is 4-bit quantization of that merged model. That step comes from quantization, not from LoRA.

Why the adapter is tiny, worked out: TinyLlama has 22 layers, hidden size 2048, MLP size 5632 and key/value size 256. A rank-16 adapter on a matrix of shape $d_{out} \times d_{in}$ adds $16 \times (d_{in} + d_{out})$ parameters:

MODULE	SHAPE	LORA PARAMS
q_proj, o_proj	2048 × 2048	65,536 each
k_proj, v_proj	256 × 2048	36,864 each
gate, up, down	5632 ↔ 2048	122,880 each
Per layer / × 22 layers		573,440 / 12.6 M

12.6 M parameters × 2 bytes = 25.2 MB, which matches the 25.3 MB file. That is about **1.1%** of the model's 1.1 B parameters.

What LoRA "ignores":

- No data is skipped. The model trains on every example.
- The **base weights stay frozen**: embeddings, layer norms and the output head are not trained.
- The update is restricted to **rank 16** per matrix. It can shift behaviour along a few directions, but it can't rewrite everything a full fine-tune could.

Does accuracy drop?

- **From the literature:** LoRA matches full fine-tuning on many tasks (Hu et al.), and QLoRA matched 16-bit fine-tuning on their benchmarks (Dettmers et al.). A persona and speaking style is a small change, which is where low rank works well.
- **Our own evidence (archived medieval run):** the full fine-tune had the strongest raw persona (46.8% dialect-marker rate) but also the worst over-insertion (score 0.8932), and was 11.86 GB. That is a trade-off, not a clear win either way.
- **Not measured:** LoRA vs full fine-tune on the modern-city data, and f16 vs Q4_K_M quality. Say this plainly if asked.

Q4

Why rank 16, alpha 32, and all seven projection modules?

SHORT ANSWER

An earlier $r=8$ on q/v only config barely changed the voice. We widened to $r=16$ on all attention and MLP projections (still only 25 MB), with $\alpha = 2 \times r$, a common scaling choice.

Config: `training/configs/lora_config.yaml`, with $r=16$, $\text{lora_alpha}=32$, dropout 0.05, bias none, and target modules q, k, v, o, gate, up, down.

The archived medieval runs tried six configs ($r=8/16/32$, $\alpha=16/32$, 3 or 8 epochs). The lesson was that **what the training data contains mattered more than rank or alpha**: training only on the dialect-heavy entries raised the marker rate from 4.7% to 14.0%, while changing hyperparameters barely moved it.

Q5

What is GGUF / Q4_K_M, and why llama.cpp instead of PyTorch?

SHORT ANSWER

GGUF is llama.cpp's model file format. Q4_K_M is a 4-bit grouped quantization that keeps some sensitive layers at higher precision. With it, a reply takes 411–449 ms on average on a laptop CPU, which meets our 500 ms real-time target.

- The PyTorch + PEFT path (`backend/main.py`) has a ~9.5 s cold load and takes seconds per reply, too slow for a game.
 - **Latency** (40 tokens, CPU, $n=5$ per run): 1186 ms mean on the old 8-core laptop with an MX450 (fail); 411–449 ms mean, 312–331 ms median on the 20-core laptop with an RTX 5060 (pass).
 - **In the playable game**: 331–333 ms median warm generation.
 - **Trade-off**: the served models are *merged* per character, so switching character swaps a whole 668 MB model. It isn't a `LoRA set_adapter()` call. The server keeps the 3 most recently used models loaded.
-

Q6

Why TinyLlama-1.1B and not GPT-4 or a 7B model?

SHORT ANSWER

Games need replies that are fast, free per line, and work offline on the player's machine. Hosted models are slow and cost money per call; 7B models don't reach 500 ms on a laptop CPU. We wanted to test what a model that *does* fit can do, and where it breaks.

TinyLlama is Apache-2.0 licensed and is the same base used by the closest prior work (Braas & Esterle, arXiv:2511.10277). A second base, Qwen3-0.6B, is planned but **not trained**, so every result so far comes from one base model.

DATA

Q7

What dataset did you use, and where did it come from?

SHORT ANSWER

Our own 1,160-pair modern-city NPC dialogue set. Two-thirds comes from SODA (allenai/soda on Hugging Face), picked by speaker role; the rest we wrote by hand to teach refusals and correct confirmations.

SOURCE	PAIRS	WHAT IT IS
SODA (allenai/soda)	766	Dialogues whose speaker is a matching role ("Officer", "Pharmacist"...)
Hand-authored refusal training	195	The NPC says it doesn't know something outside its role
Hand-authored contrastive confirm	140	The NPC <i>does</i> answer what its role knows, to fix over-refusal
Hand-authored pilot	59	First police officer batch
Total	1,160	

Per character: police officer 325, pharmacist 140, professor 120, and 115 each for shopkeeper, bartender, social worker, executive and service worker. The imbalance is deliberate: more data went where the knowledge-leak results were worst.

Also downloaded but not used:

- **MultiWOZ** (no police dialogues when we checked), **Taskmaster** and **Synthetic-Persona-Chat**, because SODA alone covered all eight roles.
- An earlier medieval dataset (Gutenberg plays, chimbiwide/NPC-Dialogue_v2) is archived and is **not a project result**.

Also built: a knowledge base of 17 world facts (2–3 per character), each with a list of characters allowed to know it and 39 signature phrases. The KBD metric (Q12) uses it.

Q8

How did you process it?

SHORT ANSWER

We matched SODA speaker names to our eight roles with patterns, took each (previous line, role's reply) as an input/output pair, and stored it in one schema with persona, context and where it came from. Then we added hand-written refusal and confirm examples, and set aside a 15% held-out split, stratified by character and seeded.

1. **Find role speakers** (`data/scripts/soda_extractor.py`): a pattern per character over SODA's speakers field. Real matching rows before filtering: executive 50,252 ... pharmacist only 99.
2. **Make pairs:** the preceding turn becomes the player's line, the role's turn becomes the NPC reply. SODA is already natural modern English, so this is extraction and relabelling, *not* rewriting, and each entry's `conversion_note` says so.
3. **Schema v1.0** per entry: id, input, output, persona (archetype, disposition), context (location, time, world state), linguistic markers, metadata (intent, source, quality score, tags).
4. **Fix what the first results showed:** the knowledge-leak tests found the adapters agreeing with leading questions, so we added refusal examples. That caused over-refusal, so we added contrastive "confirm" examples.
5. **Split** (`training/make_split.py`): 987 train / 173 held-out, 15% per character, seed 20260910. Rerunning the script produces a byte-identical file.

Q9

How did you train it?

SHORT ANSWER

One QLoRA adapter per character on TinyLlama-1.1B: 3 epochs, learning rate 2e-4, effective batch 16, bf16 compute, on a laptop GPU. Each adapter is 25.3 MB.

SETTING	VALUE	WHY
Method	QLoRA, NF4 4-bit base, double quant	Fits in laptop GPU memory
Epochs / LR	3 / 2e-4	Standard LoRA starting point
Batch	4 × 4 accumulation = 16	Memory limit
Precision	bf16 (not fp16)	fp16's loss scaler crashed on TinyLlama's bf16 layers
Held-out runs	cosine schedule, 10% warmup, eval every 2 steps	Added once the split existed
Checkpoints	every 15 steps	A GPU driver reset once lost an hour of training
Trainer	HF PEFT + TRL SFTTrainer	

Also trained: for the game, a fine-tuned GLiNER-small (166 M parameters) that pulls facts out of what the player says. It trained on 6,000 generated lines in 3 minutes on the RTX 5060, and a small intent classifier sits on top of its encoder.

Q10

How did you test it?

SHORT ANSWER

Four kinds of test: knowledge-leak probes written separately from the training data (KBD), persona drift (PDM v2) and BERTScore against the stock model, latency on real hardware, and whole-conversation tests inside the game pipeline. Design decisions used dev sets; test sets were written before the runs.

- **Knowledge probes:** for each character, questions about facts it must *not* know (forbidden) and facts it should (control). These were written separately, not drawn from the training

data.

- **Four conditions compared:**
 - A: base model with no adapter
 - B: base + adapter
 - C: base + facts in the prompt, same token budget
 - D: GPT-4o (**not run**)
- **Held-out loss** on the 173-pair split, for retrained characters.
- **Game pipeline:** scripted two-visit conversations, fact questions, memory recall, and the news-spreading test, with dev/test sets fixed before runs.

Be upfront about this:

- The 8 deployed adapters were trained *before* the held-out split existed.
- PDM v2 and BERTScore were computed on prompts from their own training data, so they are not generalization results.
- Retraining all 8 on the split is listed as pending work.

For the police officer, held-out loss bottomed at 1.7105 at step 34 of 54 and ended at 1.7425, so the deployed adapter likely trained past its best point.

Q11

What is the accuracy of the model?

SHORT ANSWER

A dialogue model has no single accuracy number, so we report a metric for each claim. The headline results: adapters beat the stock model on BERTScore (0.853 → 0.905) and persona drift for all 7 characters in the PDM v2 run. But they leak knowledge: 5 of 14 forbidden probes were confirmed. The system parts that *are* classifiers score high: fact extraction 41/42, intent 54/55.

WHAT	RESULT	READ IT AS
BERTScore F1, stock → adapter	0.853 → 0.905	Higher is better; the adapter wins for all 8 characters (training-set prompts)
PDM v2 persona drift	7/7 correct order, mean gap 0.066	Lower drift with the adapter (training-set prompts)

WHAT	RESULT	READ IT AS
KBD forbidden probes violated	5/14 (36%)	Lower is better ; 2 characters leaked on every probe
KBD control probes	3/3 correct	No false alarms
Latency, 40 tokens	411–449 ms mean	Under the 500 ms target
Fact questions in game	31/50 → 42/50	All facts in the prompt vs our retrieval + retry; refusals 7 → 0
Fact extraction from player lines (GLiNER)	41/42 test, 0 wrong facts	Zero-shot base model: 17/42
Intent classification	54/55 test	Small hand-written test set; the one miss reads "thanks a lot" as a farewell
Memory slots in conversation tests	dev 20/20 · test 25/25 · test2 20/25	test2 missed "it" as a department for all 5 NPCs
News passed on / leaked	9/9 shared · 0/9 leaked	Every share needed a guard restart
Adapter routing (pick the character from the player's line)	47.4% (91/192)	Weak; not in the paper, and not our "accuracy"

If asked for one sentence: *"The adapters sound in character and run in real time, but a 1.1B model still leaks knowledge under leading questions. Measuring that is our main contribution."*

Q12

What is new here?

SHORT ANSWER

Knowledge Boundary Drift (KBD): a per-turn metric that needs no judge model and measures whether an NPC says something its character shouldn't know, naming the exact fact it leaked. Using it, we found that blending two character adapters blends their *voice* but not their *knowledge limits*.

- **C1 · KBD:** the share of factual references in a reply that fall outside the character's visibility set (the facts it may know). It is cheap enough to run on every turn during play.
- **C2 · Adapter blending (α -sweep):** merge two adapters at weights $\alpha = 0 \dots 1$. In 2 of 3 pairs the blend leaked as much as, or more than, the *leakier* parent, while voice (PDM v2) stayed nearly flat.
- **C3 · Dataset:** the 1,160-pair modern-city set, with visibility lists on each fact.
- **Trained weights vs prompt:** putting facts in the prompt (Condition C) leaked on 4/4 probes.
- **Playable integration:** a 3D campus game with player memory, spreading news and voice, all local.

WHY & IMPACT

Q13

Why did we take this project?

SHORT ANSWER

NPC dialogue is stuck between scripted dialogue trees, which break when a player says anything unplanned, and cloud LLMs, which are slow, cost money per line and need the internet. Small local models could fill the gap, but nobody measured the failure that matters most for game worlds: characters knowing things they shouldn't.

- A shopkeeper who knows the details of a police investigation breaks the story. Existing drift metrics measure *tone*, not *knowledge*.
- Prior work on small NPC models (Braas & Esterle) looked at fluency and context retention, not knowledge limits.
- It had to be doable with student hardware: one laptop GPU, small models, no paid APIs.

Q14

Where will this project and research have impact?

SHORT ANSWER

In games, as offline, low-cost NPCs that can run on a player's own machine. More broadly, anywhere a small AI must stay within what its role is allowed to know or say.

- **Game studios, especially indie:** a 25 MB file per character and no per-line API cost. Works offline and on consoles.
- **Training simulations:** police, pharmacy, counselling or customer-service role-play, where a character must not reveal what it wouldn't know.
- **Educational tools:** campus guides and tutors that answer from a fixed fact base and say "I don't know" otherwise.
- **Role-restricted assistants:** KBD is a general check for "did the bot reveal information outside its visibility set", useful for privacy and access-scoped chatbots.
- **Research:** a judge-free metric that runs every turn, and evidence that blending adapters does not preserve knowledge limits. Anyone merging adapters should know that.

THE GAME

Q15

How does an NPC remember the player and understand what they say?

SHORT ANSWER

A fine-tuned span model (GLiNER) reads each line for facts like name, year, department and section, and each NPC saves them with a confidence score. A classifier decides what the player is doing (greeting, question, recall, report). The prompt then gets exactly one relevant piece of context, and a reply checker fixes known failure types with at most one regeneration.

- **Why we replaced the pattern rules:** "i am class cse d" renamed the player "Class". The fine-tuned model reads sentence structure instead of guessing what a name looks like.
 - **No test leakage:** the training values (names, towns, departments) are kept separate from the test lines, and the data generator refuses to run otherwise.
 - **Each NPC's memory is its own:** an NPC that was never told your name doesn't know it (0/16 in isolation tests).
 - **Other NPCs:** each NPC can point you to the other guests ("where's the officer?" → "Officer Reyes, the campus liaison officer with the city police"). Listing them in the prompt alone didn't work, so they are retrieved like facts.
 - **Pipeline:** understand → learn → retrieve → compose → generate → check. See `Docs/NPC_DIALOGUE_ARCHITECTURE.md`.
-

Q16

How do news and speech work?

SHORT ANSWER

If you report an incident to one NPC, the game stores it, and every 45 seconds one NPC who knows it tells one who doesn't. Players can talk by holding a key: Whisper base.en transcribes the audio on the server, and Piper speaks the NPC's reply.

- **News:** an NPC that hasn't heard an event never gets it in its prompt. The event is still sent to the server so a leak can be flagged, which makes this KBD for rumours.
- **Speech input:** hold Tab by default (rebindable, and the mic is selectable in F2 settings). About 140–370 ms per line on the GPU. Silence is never sent to the model, and misheard names are a known issue.

HARD QUESTIONS

Q17

What are the limitations?

SHORT ANSWER

Metrics are automatic only, with no human study. Probe counts are small, results come from one base model, and the deployed adapters were trained without a held-out split. Our test sets were written by the same team that built the system.

- **Small probe sets:** 2 forbidden probes per character; 8 per cell in the blending sweep.
 - **Dependence on repairs:** the 1.1B model often needs the reply checker. Passing on news worked 18/18 times, but *only* with a restart.
 - **Unclean comparison:** the blending rerun changed probes and retrained adapters at the same time.
 - **Loose scoring:** recall and grounding scores are keyword-based and slightly overstate results (a hand audit found 48 of 54 retried replies faithful).
 - **Pending:** held-out retrain of all 8, Qwen3-0.6B, multi-turn drift curve, GPT-4o condition.
-

Tough follow-ups to rehearse

THEY ASK	ANSWER WITH
Why not RAG instead of fine-tuning?	We tested it as Condition C: facts in the prompt at the same token budget leaked on 4/4 probes. Adapters carry the voice; facts still need retrieval with checks, which is what the game does.
Isn't 1,160 pairs too small?	For style, small and curated is enough: Braas & Esterle found 115 curated pairs beat 564 synthetic ones. And in our medieval runs, what the data contained mattered more than rank.
How do you know KBD is right?	We calibrated it on real generations: the first version flagged only 1/14 forbidden probes, and the corrected version found 5/14, with 3/3 controls correctly left unflagged. It can still miss paraphrases, which we list as a limitation.
Why is routing accuracy so low?	Player lines are mostly generic ("how do I get to X?"). What tells characters apart is in the NPC's reply, so we pick the character from the game world, not from the text.
Can it run without a GPU?	Yes. NPC replies run on CPU (n_gpu_layers=0). The GPU is only used for speech-to-text, which also runs on CPU, just slower.
What if two players tell different NPCs different things?	Memory is per NPC and news is per event, so each NPC only knows what it heard. The game owns that state and the server stores nothing between requests.
Why not merge all characters into one adapter?	That is the blending question: merged adapters lose the knowledge boundary. Separate adapters keep it measurable.
Did you use ChatGPT/Claude for data?	The modern set is SODA plus hand-authored examples. A GPT-4o augmentation script exists but was not used for this dataset. Say which team members wrote the hand-authored batches.

Things not to say (they're wrong or out of date)

- **"A full fine-tune on the same data is 11.86 GB."** That figure is from the *archived medieval* run; the paper's wording should be fixed. Say "our archived full fine-tune was 11.86 GB".
- **"LoRA made the model 2 GB."** The 2 GB is the merged 16-bit model. LoRA makes the per-character file 25 MB, and quantization brings the model to 668 MB.
- **"The dataset is 600 / 960 pairs."** Those counts are stale in older docs. It is 1,160.
- **"Accuracy is 47%" or "95%+."** 47.4% is adapter routing, a side result. There is no single model accuracy.
- **"We tested on Qwen3" or "compared with GPT-4o"**. Neither has been run.
- **"PDM/BERTScore show generalization."** They were measured on training-set prompts.
- **"Medieval results"** are archived methodology evidence, not project findings.

Sources: `paper/NUMBERS.md`, `paper/main.tex`, `training/configs/`, `training/train_adapter.py`, `data/processed/modern_npc_dataset.json`, `data/processed/modern_split.json`, `evaluation/results/`. Model sizes read from files on disk on 15 Sep 2026.